



SparseDitto: An Agentic Sparse Compilation Framework through Architecture-Aware Synthesis on GPUs

Shiyang Li
li004074@umn.edu
University of Minnesota
Minneapolis, MN, USA

Guangyan Sun
sun01158@umn.edu
University of Minnesota
Minneapolis, MN, USA

Jinwei Tang
tang0940@umn.edu
University of Minnesota
Minneapolis, MN, USA

Yanzhi Wang
yanz.wang@northeastern.edu
Northeastern University
Boston, MA, USA

Mingyi Hong
mhong@umn.edu
University of Minnesota
Minneapolis, MN, USA

Caiwen Ding
dingc@umn.edu
University of Minnesota
Minneapolis, MN, USA

Abstract

Sparse matrix computation performance on GPU depends on how representation and execution schedule match the input structure and target hardware. No single implementation consistently dominates across sparsity patterns, operators, and hardware. Existing sparse compilers and specialized systems cannot cover all of them simultaneously.

We present SparseDitto, an agentic sparse compilation framework for sparse matrix computation on GPUs. It jointly synthesizes representation, execution schedule, and hardware mapping in a unified compilation plan. Structural analysis and a learned template-ranking prior guide architecture-aware synthesis. LLM-guided lowering realizes each plan as CUDA code, while target-GPU profiling drives plan refinement. SparseDitto covers multiple operators, e.g., SpMV, SpMM, and SpGEMM, and various representations within one framework. It can also automatically adapt to different hardware. Across various SuiteSparse matrices, SparseDitto achieves geometric-mean speedups over cuSPARSE of $2.68\times$ on an NVIDIA RTX PRO 6000 and $2.79\times$ on an NVIDIA H200 (up to $146.61\times$). Its generated SpMM kernels accelerate full-batch GCN training by up to $3.39\times$.

Keywords: sparse matrix computation, GPU kernel synthesis, sparse compilation framework

1 Introduction

Sparse matrix operations are core building blocks in scientific computing, graph analytics, and machine learning [12, 16, 24, 28, 29]. Their performance on GPUs can change by orders of magnitude even when the matrix, operator, library, and GPU remain unchanged. For one matrix with 355 K nonzeros, cuSPARSE completes SpMM in $37\ \mu\text{s}$ using CSR but requires 13 ms using Blocked-ELL. Blocked-ELL regularizes sparse computation by storing each block row with a common number of blocks, which can expose efficient blocked execution. On this irregular matrix, however, the required

padding makes much of the transferred data useless and removes that advantage. The $350\times$ performance difference is caused only by the representation and its associated kernel. This example exposes a basic challenge for sparse matrix computing on GPU. A high-performance sparse implementation needs to match its representation and execution strategy to the structure of the input.

Further fundamental challenge is that sparse GPU performance depends on three coupled implementation choices (Figure 1a). The sparse representation determines the layout of values and metadata. The execution schedule determines workload partitioning, thread mapping, and data movement. Hardware mapping further binds these choices to tile sizes, launch configurations, and on-chip resources on the target GPU.

Current state-of-the-art (SOTA) approaches follow three main directions. First, specialized sparse libraries and kernels encode expert-designed optimizations for particular operators, sparsity patterns, or hardware features [4, 7, 30]. Second, sparse compilers and design-space search systems expose broader choices of representations and schedules [6, 12, 31]. Third, using LLM-based CUDA programming agents to generate and optimize GPU kernels through iterative coding, compilation, profiling, and refinement [15, 22, 33]. These directions provide optimization mechanisms, but they partially addressed the challenges. The key missing capability is to coordinate sparse-specific representation and scheduling decisions with flexible code construction and target-specific hardware adaptation.

We study three representative operator-specialized systems and one compiler-based system: CB-SpMV [4], DTC-SpMM [7], HSMU-SpGEMM [30], and SparseTIR [31]. We obtain two key observations. **First, sparse optimization is highly asymmetric.** A compatible representation and schedule can provide meaningful gains, while a poor match can cause orders-of-magnitude slowdowns. The best alternative format improves SpMV by up to $1.47\times$ and SpMM by up

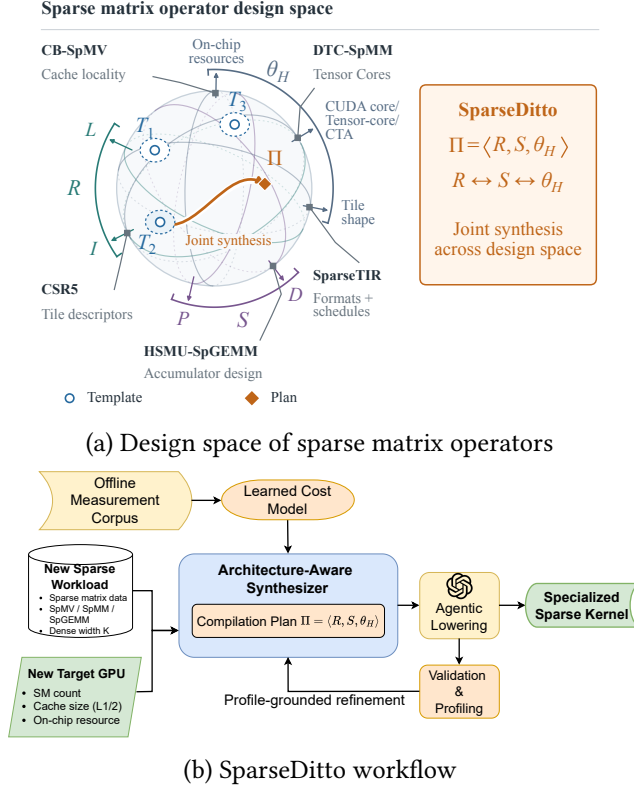


Figure 1. SparseDitto’s design space and compilation workflow. Prior-work annotations in (a) highlight representative mechanisms.

to 2.8×, while a poor pairing loses up to 350×. **Second, no fixed implementation consistently dominates across sparsity patterns, operators, and GPUs.** CB-SpMV works well on cache-friendly blocked inputs. DTC-SpMM benefits matrices that keep enough useful work inside compressed Tensor-Core blocks. HSMU-SpGEMM performs well when its hash accumulator matches the intermediate-product distribution. Their relative performance also changes with the GPU architecture and CUDA runtime version.

The three SOTA directions leave complementary gaps. Specialized kernels cover only the sparsity patterns, operators, and hardware assumptions for which they were designed. Sparse compilers and design-space search systems broaden the optimization space, but their coverage is still bounded by developer-provided static rules. Search can also be expensive. AlphaSparse [6], for example, can spend hours constructing a specialized SpMV implementation through offline search. General CUDA programming agents offer more flexible code construction, but they lack sparse-specific guidance for choosing representations and execution schedules. A functionally correct kernel can therefore remain inefficient when these high-level decisions do not match the input

structure. A general solution needs both structured sparse optimization and the ability to adapt to the target GPU.

Our target is to end the game of designing a separate sparse optimization for every limited use case and hardware platform. We seek a framework that can combine existing sparse optimization strategies, use each where it fits, and adapt to new GPU platforms with minimal additional human effort. This goal leads to three requirements. The framework must jointly reason about representation, execution schedule, and hardware mapping. It must use structural analysis to guide these choices. It must also incorporate target-GPU measurements when static analysis is insufficient.

In this paper, we present **SparseDitto**, the first agentic sparse compilation framework that automatically specializes GPU kernels for a given sparse matrix, operator, and target GPU. As shown in Figure 1b, SparseDitto treats sparse kernel construction as a compilation problem. Architecture-aware synthesis builds a unified plan for representation, execution schedule, and hardware mapping. Agentic lowering realizes the plan as a specialized CUDA implementation for repeated execution. Subsequent calls reuse it for the same sparsity structure and operator configuration on the target GPU.

In summary, this paper makes the following contributions:

- **A unified sparse kernel compilation framework on GPUs.** We formulate sparse GPU kernel compilation as the joint choice of data representation, execution schedule, and target-specific hardware mapping. The formulation captures the major implementation decisions shared by SpMV, SpMM, and SpGEMM within one framework.
- **Structure-guided and architecture-aware synthesis.** A structural analysis computes 36 features that describe the sparsity pattern, representation overhead, and operator-dependent intermediate work. An interpretable additive energy model serves as a learned cost model and ranks established optimization templates from an offline measurement corpus. An architecture-aware synthesis stage combines this ranking with static workload bounds and target-GPU constraints to construct candidate compilation plans.
- **LLM-guided lowering with profile-guided optimization.** SparseDitto lowers each compilation plan into CUDA using an LLM-guided coding stage and a set of supported sparse optimization mechanisms. Compilation and numerical validation check each candidate. Runtime measurements and profiling on the target GPU guide later changes to the representation, schedule, and hardware mapping. The fastest valid implementation is retained for reuse.

We evaluate SparseDitto on 60 SuiteSparse matrices, covering SpMV, SpMM at four dense widths, and SpGEMM. SparseDitto reaches a geometric-mean speedup of 2.68× over cuSPARSE on an RTX PRO 6000, with a maximum of 146.61×.

On an H200, it reaches 2.79 $\times$, with a maximum of 78.5 $\times$. The specialized SpMM kernels also accelerate full-batch GCN training by up to 3.39 $\times$.

2 Background and Motivation

2.1 Sparse Matrix Operators on GPUs

The sparsity pattern determines the distribution of nonzeros, the memory a kernel traverses, and the workload assigned to computing units. It also controls locality once the schedule is mapped to the GPU. The operator determines the work produced by each visited nonzero. As a result, the critical design choice changes across SpMV, SpMM, and SpGEMM.

SpMV computes $y = Ax$ and performs only two floating-point operations per nonzero. Its performance often depends on sparse metadata traffic and the locality of gathers from x . Load imbalance among parallel units is another major cost [8, 16, 17]. Row-based CSR avoids output reductions but inherits the row-length distribution. ELL and SELL use padding to regularize sparse traversal.

SpMM computes $C = AB$ with a dense matrix B of width K . Each nonzero of A drives K multiply-adds, which creates another pattern dimension and more reuse of B . The sparse representation needs to be coordinated with the mapping of rows or blocks to the GPU. The K -tile and output-accumulator placement also affect the dataflow. Dense sparse tiles can enable Tensor Cores when useful arithmetic outweighs compaction and fill-in costs [7, 29].

SpGEMM further discovers a sparse output. It computes $C = AB$ for two sparse matrices inputs A and B . In a row-wise algorithm, row i generates

$$u_B(i) = \sum_{k \in A[i,:]} nnz(B[k,:]), \quad (1)$$

The distribution of $u_B(i)$ affects row partitioning and temporary storage. It also determines the suitability of dense, hash, merge, or hybrid accumulators [18, 27, 30]. Design space now extends beyond input traversal because the kernel must construct output indices and intermediate state.

Thus, the same pattern can produce different metadata streams, processing units, and padding or fill costs. Figure 2 shows these effects on a common matrix. They are coupled to the parallel decomposition selected by the GPU kernel.

These interactions motivate two questions. First, with the operator and GPU fixed, how strongly does the profitable implementation depend on the sparsity pattern? Second, how much of this pattern-dependent optimization space is covered by existing specialized systems and sparse compilers?

2.2 Sensitivity to Sparsity Patterns

We first hold the library and platform fixed to cuSPARSE [20], then vary the representation and SpMM width. The top panel of Figure 3 uses format names for brevity, but every bar measures a complete implementation.

The observed spread follows the assumptions of each implementation. SELL-C wins by up to 1.47 $\times$ on a stencil with only 0.08% slice padding. It loses 33 $\times$ when uneven row lengths inflate storage by 30 $\times$. BSR wins 1.25 $\times$ when occupied 16 $\times$ 16 tiles are 87% dense. It falls to 0.04 $\times$ on power-law graphs because the occupied blocks contain mostly zeros. COO pays for explicit row metadata and reductions on regular inputs. Under row imbalance, its nonzero-level decomposition makes it the most robust SpMM choice. COO wins 21 of 30 configurations and reaches 2.8 $\times$. Across the full sweep, the best alternative gains at most 2.8 $\times$ over CSR. A mismatched design loses up to 350 $\times$.

The operator context changes the same tradeoff. At $K=32$, block-dense matrices can favor CUDA-Core blocked kernels. At $K=128$, Tensor-Core layouts replace them on some inputs. Blocked-ELL accelerates dense-tile arithmetic but pads every block row to a common width. This padding can increase transferred bytes by two orders of magnitude. For SpGEMM, the corresponding choices appear in row bucketing and phase structure. The accumulator dataflow also depends on the intermediate-work distribution [18, 19, 23, 30].

Observation 1. The profitability of a sparse optimization depends strongly on the input structure. Representation and execution schedule must therefore be coordinated rather than chosen independently. A poor match can be far more costly than the gain from a good one.

2.3 Limitations of Existing Works

We next evaluate three recent operator-specialized systems and one sparse compiler on the same NVIDIA RTX PRO 6000 GPU platform. CB-SpMV [4] combines cache-friendly blocks with adaptive per-block formats. DTC-SpMM [7] combines compressed block metadata with Tensor-Core execution. HSMU-SpGEMM [30] uses a shared-memory hash accumulator for SpGEMM. SparseTIR [31] supports dynamic format decomposition and scheduling for SpMM.

Figure 3 shows how they perform. CB-SpMV achieves 1.7–2.5 $\times$ while its blocked working set remains cache-friendly. It loses on every matrix above 3 M nonzeros and falls to 0.32 $\times$. DTC-SpMM reaches 3.1 $\times$ on block-structured inputs. It also beats cuSPARSE’s padded Tensor-Core format by 26–106 $\times$ on irregular matrices. However, under extreme row imbalance DTC-SpMM falls to 0.04 $\times$. HSMU-SpGEMM surpasses cuSPARSE on three and loses on seven of ten matrices. None of the three systems dominates the full set.

The target platform changes the useful region further. These systems were developed for earlier GPU and CUDA generations. On Blackwell and CUDA 13.0, their fixed resource choices meet different hardware conditions, and they adapt poorly to the new platform. DTC-SpMM, for example, reports more than 1.5 $\times$ over cuSPARSE on an RTX 4090 [7]. Running the released artifact on our Blackwell GPU with CUDA 13.0, we measure it below cuSPARSE at every dense width, from 0.73 $\times$ at $K=8$ down to 0.58 $\times$ at $K=256$.

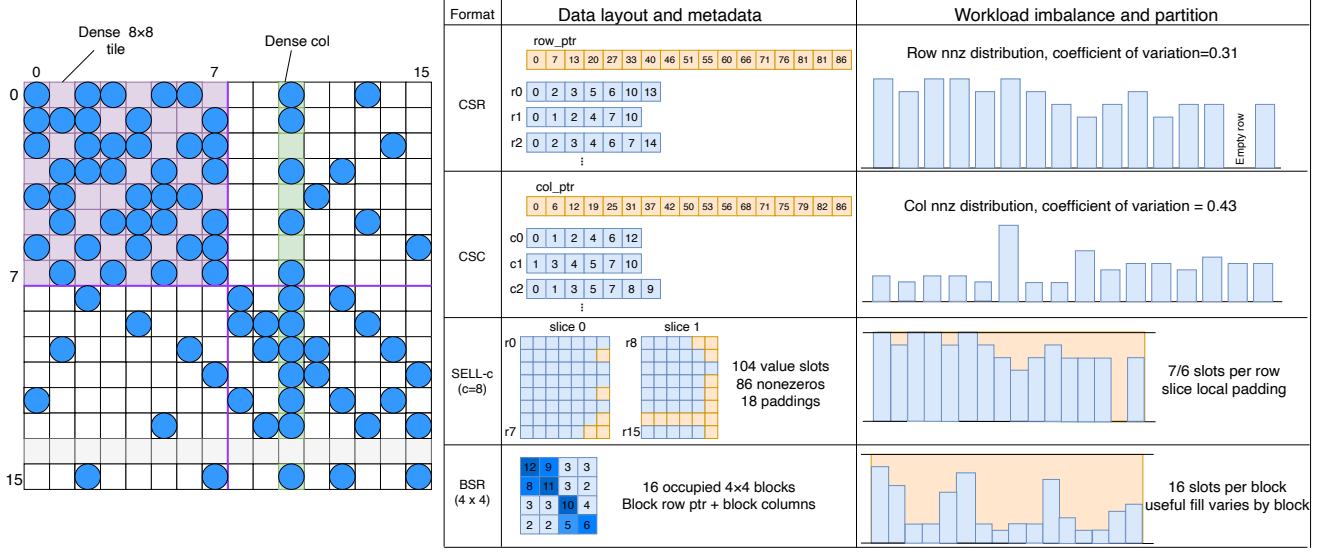


Figure 2. One 16×16 matrix exposes intrinsic pattern cues and format-induced costs. The matrix contains an irregular dense tile, a high-degree column, an empty row, and uneven row lengths. CSR and CSC traverse compressed rows and columns, respectively. SELL-C with $C = 8$ introduces slice-local padding, while 4×4 BSR introduces block fill. The right column shows the resulting work granularity and imbalance for rows, columns, slice slots, and blocks.

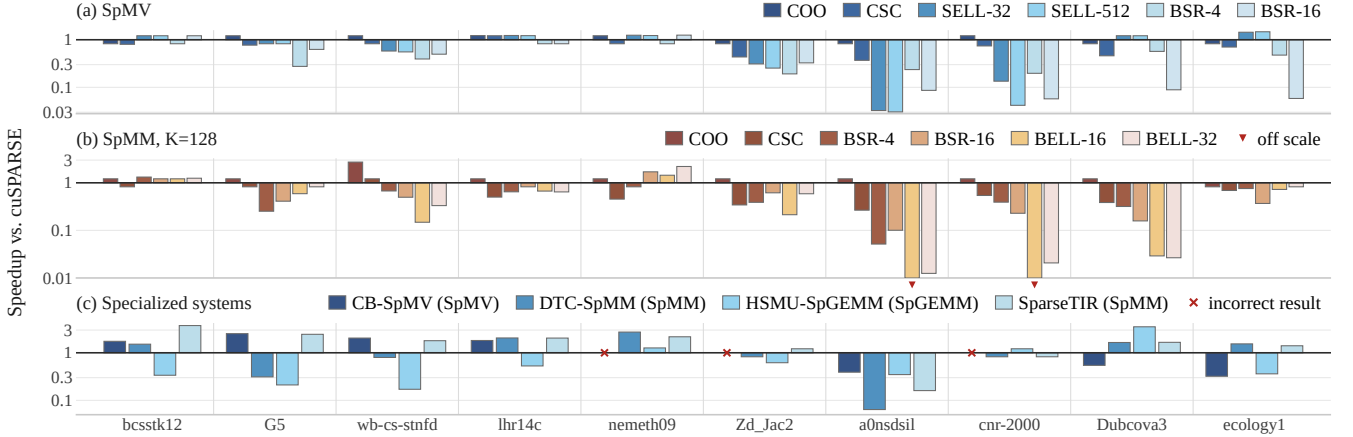


Figure 3. Sparse kernel performance sensitivity on ten SuiteSparse matrices [5]. Top: cuSPARSE implementations for different sparse representations in SpMV and SpMM at $K=128$. Bottom: three specialized systems and one sparse compiler. The best design changes with the pattern, while mismatched choices create severe slowdowns. All measurements use an NVIDIA RTX PRO 6000 Blackwell GPU with CUDA 13.0 and report the mean of ten timed iterations after warmup.

Sparse tensor compilers expose a broader space through dynamic format decomposition. They also provide scheduling primitives. SparseTIR [31] beats cuSPARSE on 25 of 30 SpMM configurations. Its geometric-mean speedup is $1.40\times$ and its maximum is $5.0\times$. SparseTIR nevertheless falls to $0.13\text{--}0.20\times$ on the matrix behind the $350\times$ format cliff because long rows serialize execution. SparseTIR can express a bucketed hybrid layout, but a developer supplies the format rewrite and schedule [31]. Its coverage therefore depends on the implementations encoded in its rule set. It also does not

support SpMV or SpGEMM, and it covers only a limited set of widths ($K = 32, 128, 256$) for SpMM.

Observation 2. No fixed implementation or developer-defined optimization space consistently covers different sparsity patterns, operators, and GPU architectures. The useful optimization region changes with both the input structure and the target platform.

Together, Observations 1 and 2 show that sparse kernel optimization is both pattern dependent and architecture dependent. This motivates treating sparse kernel construction as a

per-workload compilation problem. Representation, execution schedule, and hardware mapping should be constructed together for the input structure and target GPU.

3 SparseDitto Design

SparseDitto treats sparse GPU kernel construction as a per-workload compilation problem. A compilation task is specified by a sparse input X , an operator o , and a target GPU H . Given $\langle X, o, H \rangle$, SparseDitto constructs a specialized CUDA implementation together with the sparse data structures required by that implementation.

SparseDitto represents the main compilation decisions with a *compilation plan*

$$\Pi = \langle R, S, \theta_H \rangle, \quad R = \langle L, I \rangle, \quad S = \langle P, D \rangle. \quad (2)$$

The representation R consists of a data layout L and sparse metadata I . The layout specifies how sparse values are organized and stored, while the metadata supports traversal of the resulting representation. The execution schedule S consists of a parallel decomposition P and a dataflow D . The decomposition maps work to Cooperative Thread Arrays (CTAs), warps, and threads, while the dataflow determines operand movement and accumulation. Finally, θ_H contains target-specific choices such as tile sizes, launch configuration, and on-chip resource allocation. These decisions are coupled. A change in any of them can impact others.

Figure 4 shows the overall compilation process. SparseDitto first analyzes the sparse input and estimates the overhead. A energy-based model then ranks reusable optimization templates. The architecture-aware synthesis stage combines this ranking with the input analysis and target-GPU constraints to construct candidate compilation plans. Finally, an agentic lowering stage realizes each plan as CUDA. Sanitizer, numerical validation, and target-GPU profiling provide feedback for subsequent plan refinement.

3.1 Structural Features and Overhead Modeling

SparseDitto performs a lightweight sparse analysis before compilation-plan synthesis. The analysis is guided by known sparse optimization mechanisms. For each mechanism, we identify the structural conditions that make it effective and estimate the overhead it can incur, e.g., extra storage. The analysis produces three groups of continuous features:

$$\Phi(X, o) = \langle \Phi_{\text{intr}}, \Phi_{\text{repr}}, \Phi_{\text{op}} \rangle. \quad (3)$$

Φ_{intr} describes the input sparsity pattern. Φ_{repr} estimates the overhead introduced by candidate sparse representations. Φ_{op} captures operator-dependent work that is not visible from the input structure alone. Together, these groups contain 36 continuous features.

Intrinsic pattern features. Let $r_i = \text{nnz}(X[i, :])$ denote the number of nonzeros in row i . Matrix size and density characterize the overall sparse working set. SparseDitto records the

mean, maximum, and coefficient of variation (CV) of r_i . It also records row-length quartiles and the fraction of empty rows. These statistics expose workload imbalance relevant to CSR5, merge-based decomposition, and SELL- C - σ [13, 16, 17].

SparseDitto also measures structure along the column dimension. Column-degree statistics capture reuse and asymmetry between rows and columns. Row span and normalized row span characterize access locality. We additionally record the mean gap between sorted column indices and the fraction of diagonal entries. To capture block structure, SparseDitto measures the coverage of occupied 16×16 tiles and the fraction of nonzeros contained in them. This group contains 18 continuous features. SparseDitto also assigns the row-length distribution to one of five shape classes: uniform, moderate, skewed, power-law, or bimodal.

Representation-induced overhead features. The representation $R = \langle L, I \rangle$ changes both the working set size and the metadata traversed by a kernel. SparseDitto computes value-slot expansion for padded and blocked layouts:

$$\begin{aligned} \rho_{\text{ELL}} &= m \max_i r_i / \text{nnz}(X), \\ \rho_{\text{SELL}}(c) &= \sum_s |s| \max_{i \in s} r_i / \text{nnz}(X), \\ \rho_{\text{BSR}}(b) &= b^2 \text{nnzb}_b / \text{nnz}(X), \end{aligned} \quad (4)$$

where s denotes a row slice and nnzb_b counts occupied $b \times b$ blocks. These factors estimate padded computation and value traffic. The storage estimate also accounts for slice offsets and block-column indices. Row permutations are included when required. SparseDitto evaluates ELL and three SELL slice heights. It also evaluates BSR with $b \in \{4, 8, 16\}$. These choices produce eight continuous representation features. Their values are recomputed when L , I , or the format parameters change during plan refinement.

Operator-induced features. Input statistics alone do not expose the intermediate iteration space of SpGEMM. From Equation 1, SparseDitto computes the total, maximum, p95, and CV of $u_B(i)$ for the requested product. It also computes the total and CV for $B = X^T$. The accumulator-fit statistic

$$F_h = \frac{1}{m} \sum_i \mathbf{1}[u_B(i) \leq h] \quad (5)$$

reports the fraction of rows below capacities $h = 512, 4K$, and $32K$. These thresholds distinguish accumulator regimes that a mean would hide. SparseDitto also computes

$$\gamma_B = \sum_i u_B(i) / \text{nnz}(C), \quad (6)$$

which measures the intermediate products merged into each output nonzero. The estimator uses stratified sampling inspired by dataflow analysis in [27]. The ten features in this group capture intermediate work, imbalance, accumulator demand, and output compression.

Hardware and task context. Target-GPU properties are kept separate from the 36 sparse features. SparseDitto records the number of SMs and physical cache capacities in bytes.

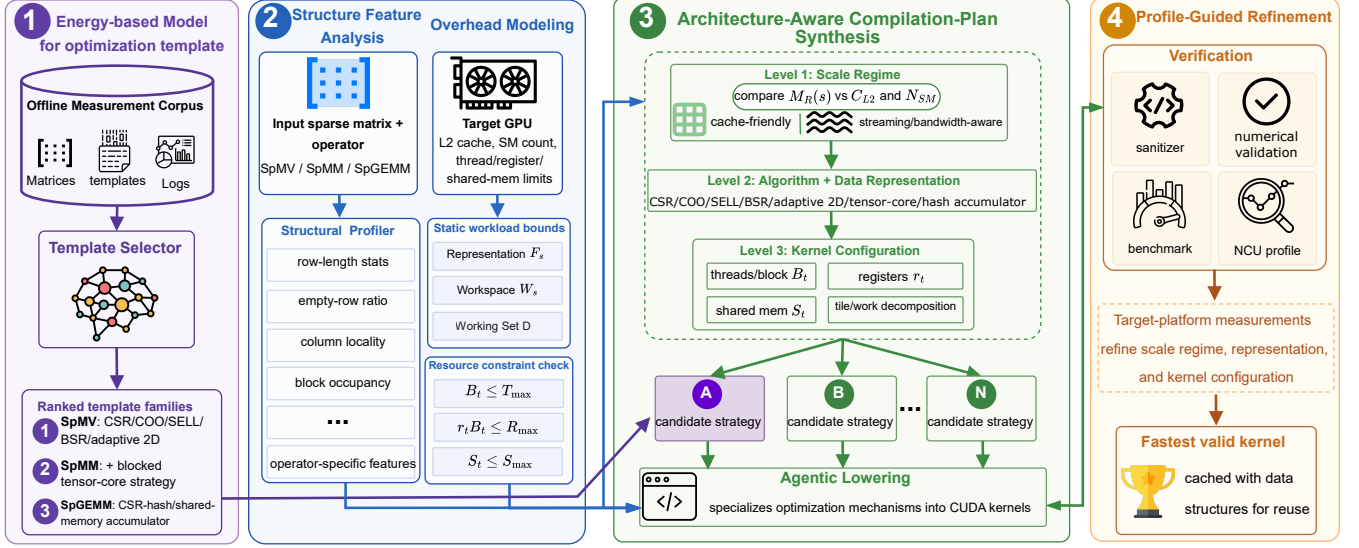


Figure 4. Overview of SparseDitto. Sparse pattern analysis and a learned cost model identify promising optimization templates. Architecture-aware synthesis constructs compilation plans that coordinate representation, execution schedule, and hardware mapping. Agentic lowering and profiling realizes these plans as CUDA code.

It also records per-CTA hardware limits on threads, registers, and shared-memory size. The operator and the dense width K for SpMM specify the task context. These values are used by the learned cost model when applicable and by the architecture-aware synthesis stage to construct and check compilation plans.

3.2 Energy-Based Model for Optimization Templates

The learned cost model provides a prior over reusable optimization templates. We adopt an additive parameterization so that each ranking can be decomposed into feature-level contributions and passed to compilation-plan synthesis. It uses measured experience to rank template set V_o , that has a large effect on representation overhead, workload decomposition, or dataflow [4, 7, 20, 25, 30–32].

For SpMV, V_o contains CSR, COO, SELL with slice heights {16, 32, 512}, BSR with block sizes {4, 8, 16}, and adaptive 2D blocking. The SpMM set contains CSR, COO, the same three BSR variants, Blocked-ELL-{16, 32}, and the DTC Tensor-Core design [7]. The three SpGEMM templates are CSR-hash, shared-memory tiled hash, and hybrid accumulation. These templates draw on established GPU kernels [13, 27].

The learned cost model is a single additive energy model [11, 14]. Its input x concatenates the 36 sparse features, a one-hot shape class, an operator indicator, and the tile width $\log_2 K$ for SpMM. For each scalar coordinate x_j , a nonlinear basis shared across operators produces

$$h_j(x_j) = \tanh(x_j w_{1j} + b_{1j}) \in \mathbb{R}^{16}. \quad (7)$$

Each operator o then has its own readout for every template $s \in V_o$:

$$E_o(s | x) = \beta_{o,s} + \sum_j h_j(x_j)^T w_{2,o,j,s}. \quad (8)$$

Lower energy indicates a stronger recommendation. Sharing the basis across operators lets a single feature response be estimated from all three tasks, while the operator-specific readouts capture the different effect of that feature on SpMV, SpMM, and SpGEMM. The nonlinear basis can express threshold behavior, such as a sharp penalty once SELL padding exceeds a level, without a hand-designed cutoff.

We train the energy directly from measured runtimes. For a training example, let $M \subseteq V_o$ be the templates actually benchmarked and $t_s = \log_2(\text{baseline runtime}/\text{runtime of } s)$ their measured log-speedups. The objective combines a listwise likelihood [3] with a gap-weighted pairwise margin [2]:

$$\mathcal{L}_o = \sum_{(x,t)} \left[- \sum_{s \in M} q_s \log p_o(s | x) + \lambda_o \sum_{\substack{s,s' \in M \\ t_s > t_{s'}}} (t_s - t_{s'}) \text{softplus}(E_o(s | x) - E_o(s' | x)) \right] \quad (9)$$

where $p_o(s | x) = \text{softmax}_{s \in M}(-E_o(s | x))$ and $q_s = \text{softmax}_{s \in M}(t_s/T)$ are soft targets from the measured speedups. The listwise term keeps near-optimal alternatives ranked highly, while the margin term orders templates by measured speed with a penalty proportional to the speedup gap. The operator-specific weight λ_o controls the strength of the margin term. It is smaller for SpMV, whose template speedups cluster more tightly, and larger for SpMM and SpGEMM.

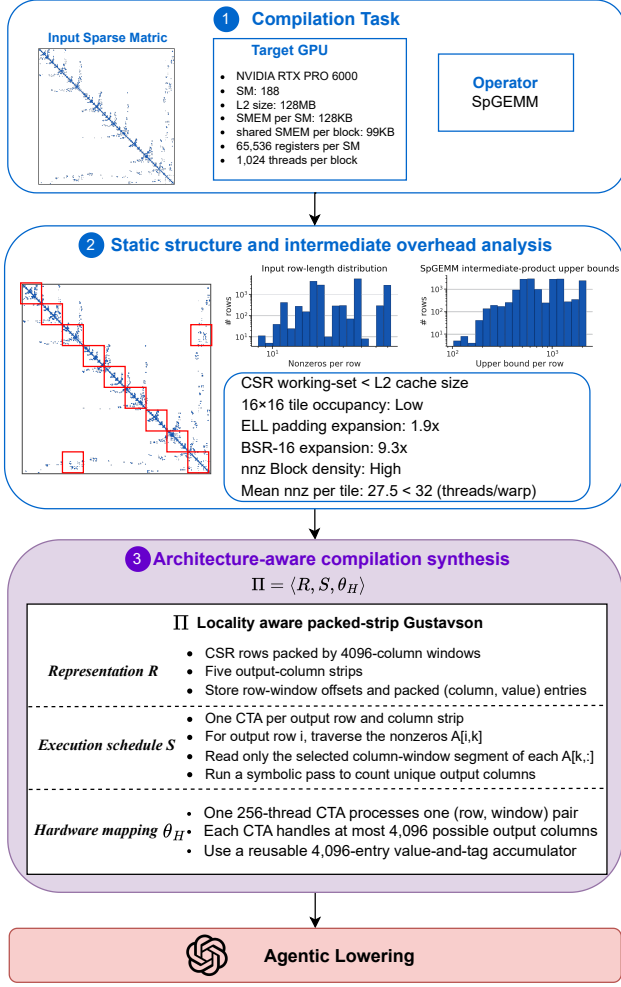


Figure 5. Architecture-aware synthesis of one SpGEMM compilation plan for Shen/e40r0100 on an RTX PRO 6000.

The additive form decomposes each score into per-feature terms $h_j(x_j)^T w_{2,o,j,s}$, so the model is interpretable by construction [1]. The synthesis stage receives these terms as feature-level explanations for the template ranking. The model returns a ranked set of templates. The ranking therefore guides the known part of the optimization space.

3.3 Architecture-Aware Compilation-Plan Synthesis

The synthesis stage combines three inputs: the sparse-analysis results, the learned template ranking, and the target-GPU specification, as shown in Figure 4. It expands promising templates into complete compilation plans. Each plan records R , S , and θ_H , together with storage estimates, representation overhead modeling, and on-chip resource bounds.

Workload-scale analysis. SparseDitto first estimates the storage and work exposed by a candidate design. Let

$$M_{\text{rep}}(\Pi) = M_{\text{value}}(L) + M_{\text{meta}}(I) \quad (\text{Bytes}) \quad (10)$$

be the representation footprint. M_{value} multiplies the number of stored value slots by the bytes per value. M_{meta} similarly accounts for the byte width of each metadata field. Let $W(\Pi)$ denote temporary storage introduced by the dataflow, also in bytes. The read-side and total working sets are

$$M_R(\Pi) = M_{\text{rep}}(\Pi) + W(\Pi) + D_R \quad (\text{Bytes}) \quad (11)$$

$$M_T(\Pi) = M_R(\Pi) + D_S \quad (\text{Bytes}) \quad (12)$$

where D_R and D_S are the byte footprints of common input and output operands. For SpMM, these terms include dense B and C . For SpGEMM, $W(\Pi)$ includes the selected accumulator and intermediate representation.

Let C_{L2} denote the physical L2 capacity in bytes. The scale ratios are

$$q_R(\Pi) = \frac{M_R(\Pi)}{C_{L2}}, \quad (13)$$

$$q_T(\Pi) = \frac{M_T(\Pi)}{C_{L2}}.$$

SparseDitto also reports the number of independent work units per SM, $N_{\text{work}}(\Pi)/N_{\text{SM}}$. A work unit can be a row, slice, nonzero partition, or tile. These quantities are static scale indicators. They do not predict cache residency or achieved parallelism. Both depend on the lowered code and are determined through target-GPU measurements.

Joint representation-schedule synthesis. The second stage uses these estimates and the learned template ranking to construct R and S together. It selects a sparse algorithm and specifies the layout, traversal metadata, parallel decomposition, and dataflow needed to realize it. For example, a SELL plan couples row slicing and slice offsets with a schedule that processes slices in parallel. A SpGEMM hash plan couples row bucketing with the accumulator dataflow. These choices determine both the work assigned to each computing unit and the temporary storage it requires.

Hardware mapping. The third stage binds R and S to target-specific parameters in θ_H , including tile sizes, launch geometry, and on-chip resource allocation. For a hardware mapping t , let B_t be its threads per CTA, r_t its registers per thread, and S_t its shared-memory bytes per CTA. Let T_{max} , R_{max} , and S_{max} denote the corresponding hardware limits. A legal mapping must satisfy

$$B_t \leq T_{\text{max}}, \quad r_t B_t \leq R_{\text{max}}, \quad S_t \leq S_{\text{max}}. \quad (14)$$

SparseDitto can generate multiple different compilation plan candidates to expand its coverage of the design space. Each completed plan starts an independent search branch. Initial 5 iterations preserve optimization diversity, while later iterations favor the fastest valid plans. Before lowering, the synthesis stage checks the requested thread count and shared-memory allocation. After compilation with `nvcc`, the reported register count completes the check.

Figure 5 illustrates the synthesis of one SpGEMM plan for Shen/e40r0100 on an RTX PRO 6000. Here, $C = A^2$, $m = n = 17,281$, and $\text{nnz}(A) = 553,562$. Scanning the CSR structure

gives $\max_i r_i = 62$ and $\text{nnzb}_{16} = 20,130$. Substituting these counts into Equation 4 gives $\rho_{\text{ELL}} = 17,281 \times 62 / 553,562 \approx 1.9$ and $\rho_{\text{BSR}}(16) = 16^2 \times 20,130 / 553,562 \approx 9.3$. The same block count gives $553,562 / 20,130 \approx 27.5$ nonzeros per occupied tile, below the 32 threads in a warp; only $20,130 / [17,281 / 16]^2 \approx 1.7\%$ of all tiles are occupied. The two histograms show r_i and $u_A(i) = \sum_{k \in A[i,:]} r_k$ from Equation 1; the latter ranges from 102 to 2,187 intermediate products per row.

With FP32 values and 32-bit indices, Equation 10 gives the CSR input footprint as $8 \times 553,562 + 4 \times (17,281 + 1) = 4,497,624$ bytes, or 4.29MB. This explains the figure’s comparison with the GPU’s 128 MB L2 capacity. The complete plan’s working sets and scale ratios additionally include its workspace and other operands through Equations 12 and 13.

These estimates guide a packed column-window design that preserves diagonal locality while avoiding dense-block padding. The 4,096-column window and 256-thread CTA are synthesis choices in θ_H . The window width yields 5 output-column strips in R , and S assigns one CTA to each row-window pair.

3.4 Agentic Lowering and Refinement

Each compilation plan is realized as executable CUDA through an agentic lowering stage. The lowering stage receives the selected representation, execution schedule, and hardware mapping from Π . Its role is to realize these decisions using target-specific CUDA code.

SparseDitto provides conversion tools and metadata builders for CSR, COO, ELL, SELL, and BSR. It also provides adaptive 16×16 blocks inspired by CB-SpMV [4]. Packed column windows for SpMM follow Sextans [26]. SpGEMM support includes upper-bound analysis and row binning. Scans and workspace management follow the row-wise structure of MatRaptor [27]. These mechanisms implement L and I while providing building blocks for P and D .

An LLM-based coding agent realizes the thread mapping, accumulator design, and θ_H parameters specified by the plan within a unified harness. Every generated implementation is compiled and numerically validated. SparseDitto then measures latency and profiles individual CUDA kernels with NCU on the target GPU. Profiling feedback is mapped back to the corresponding fields of Π . Excessive padding or metadata traffic triggers changes to L or I . Imbalance and synchronization overhead trigger changes to P . Cache behavior and workspace traffic guide changes to D . Hardware resource pressure changes θ_H . The synthesis stage updates the compilation plan before the coding agent revises the code.

SparseDitto returns the fastest numerically valid implementation across all branches. The cached result is identified by its sparsity pattern, operator, and target platform. For SpMM, the key also records K . Subsequent invocations with the same key execute the compiled implementation without repeating plan synthesis or lowering.

4 Evaluation

Workloads. We use a stratified set of 60 matrices from the SuiteSparse Matrix Collection [5]. It covers regular stencils, finite-element matrices, circuit and process-simulation problems, power-law web graphs, road networks, and gene networks, and includes the hardest cases from the evaluations of the specialized systems we compare against. For each matrix, we evaluate SpMV, SpMM with $K \in \{8, 32, 128, 256\}$, and SpGEMM. For SpGEMM, we follow [30], computing AA for square matrices and AA^T otherwise.

The matrices have 359–16,777,216 rows and 817 to 101 million nonzeros. Their densities range from 1.78×10^{-7} to 0.110. Mean row length ranges from 1.0 to 1,107.1 nonzeros, and the coefficient of variation of row length ranges from 0 to 12.2. Appendix C lists every matrix with these statistics.

Platform and baselines. We evaluate on two GPU architectures, NVIDIA RTX PRO 6000 Blackwell with CUDA 13.0 and NVIDIA H200 Hopper with CUDA 12.9. We report the main results on both. The ablation study and the cache and memory-traffic analysis run on the RTX PRO 6000. We use the corresponding cuSPARSE implementation [20] as the per-task reference. We benchmark all applicable cuSPARSE algorithms under the same numerical contract and use the fastest valid configuration as the baseline. For SpMV, we compare with CB-SpMV [4] and AlphaSparse [6], a auto-tuner that creates machine-designed SpMV formats and kernels. For SpMM, we compare with DTC-SpMM [7] and SparseTIR [31], a sparse compiler for SpMM. For SpGEMM, we compare with HSMU-SpGEMM [30].

LLM configuration. The agents used for plan synthesis, CUDA lowering, and verification call the same model: GPT-5.6-terra [21] with reasoning effort *low* and a budget of 8,192 tokens per call. All other parameters are left at the API default. Every task uses the same search budget. Three candidate plans start independent branches, each with up to five lowering-and-evaluation iterations. The two fastest valid branches receive up to five additional iterations for profile-guided refinement.

Measurement protocol. We time the generated kernel and baselines with CUDA events. Each implementation has five warmup executions followed by ten timed executions, and we report their arithmetic mean, excluding the one-time preprocessing. Aggregate speedups are geometric means over tasks, reported with a 95% confidence interval obtained by bootstrapping the tasks (20,000 resamples of the log-speedups).

Correctness validation. We use the corresponding FP32 cuSPARSE implementation as per-task correctness reference. SparseDitto and cuSPARSE receive identical input values. For SpMV and SpMM, a candidate is considered correct only if every output entry satisfies

$$\left| \hat{y}_i - y_i^{\text{ref}} \right| \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \left| y_i^{\text{ref}} \right|, \quad (15)$$

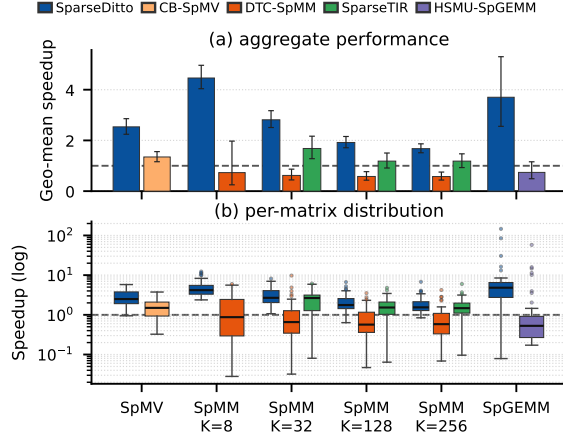


Figure 6. Generated kernel performance for SpMV, SpMM, and SpGEMM. (a) geometric-mean speedup over cuSPARSE; whiskers are 95% bootstrap confidence intervals over tasks. (b) the per-matrix distribution on a logarithmic scale. The dashed line is cuSPARSE.

where $\epsilon_{\text{abs}} = 10^{-5}$ and $\epsilon_{\text{rel}} = 10^{-3}$. Any runtime error, unexpected NaN or infinity, or violation of this bound causes the candidate to fail validation.

For SpGEMM, correctness includes both the sparse output structure and the associated values. We canonicalize the SparseDitto and cuSPARSE outputs by sorting column indices within each row and coalescing duplicate entries. The output dimensions, number of nonzeros, row pointers, and column indices must match exactly. Every corresponding nonzero value must additionally satisfy the same mixed absolute and relative error bound above. We apply this single predeclared validation rule uniformly to all matrices, generated candidates, and comparison systems.

4.1 Generated Kernel Performance

We organize the results by operator and report each one on both GPUs. On seven of the largest matrices, the cuSPARSE SpGEMM implementation triggers an out-of-resources error, so those tasks have no reference and are excluded. Each additional system is evaluated only for the operator and configurations supported by its released artifact. Aggregate speedups are geometric means over the validated tasks.

On the RTX PRO 6000, SparseDitto reaches 2.68 $\times$ over cuSPARSE (95% CI [2.48, 2.89]). On the H200, the same compilation framework reads the new target’s device properties and regenerates every implementation from scratch. It reaches 2.79 $\times$ (95% CI [2.61, 2.99]). Figures 6 and 7 break down the results by operator and dense width.

SpMV. Against cuSPARSE, SparseDitto achieves 2.53 $\times$ on the RTX PRO 6000 and 2.68 $\times$ on the H200. CB-SpMV achieves 1.35 $\times$ and 1.70 $\times$ on the two GPUs. On the same inputs, SparseDitto is 1.91 $\times$ and 1.57 $\times$ faster than CB-SpMV.

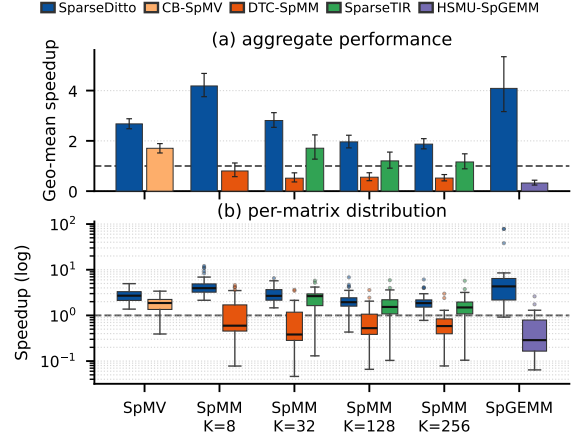


Figure 7. The same measurement on the H200. Both panels follow Figure 6.

AlphaSparse generates a customized SpMV implementation for each matrix by searching a design space defined by static rules. With a nine-hour search budget per matrix, it produces a kernel that passes our validation rule on 13 matrices. The remaining runs crash or return values outside the error bound. On those 13 matrices, SparseDitto is 5.34 $\times$ faster than AlphaSparse.

SpMM. Against cuSPARSE, SparseDitto achieves 4.46 $\times$ at $K = 8$, 2.81 $\times$ at $K = 32$, 1.92 $\times$ at $K = 128$, and 1.68 $\times$ at $K = 256$. The gain is largest at the small dense widths and narrows as K grows. The H200 follows the same trend, at 4.18 $\times$, 2.81 $\times$, 1.96 $\times$, and 1.87 $\times$.

For SpMM, we also compare with SparseTIR and DTC-SpMM. SparseTIR reaches 1.68 $\times$ at $K = 32$. Its speedup is 1.18 $\times$ at both $K = 128$ and $K = 256$. DTC-SpMM achieves 0.73 $\times$ at $K = 8$ and 0.62 $\times$, 0.58 $\times$, and 0.58 $\times$ at the three larger widths, so it stays below cuSPARSE at every width. Across the evaluated tasks, SparseDitto is 1.57 $\times$ faster than SparseTIR and 3.99 $\times$ faster than DTC-SpMM. On the H200, the corresponding speedups are 1.63 $\times$ and 4.32 $\times$.

SpGEMM. SparseDitto achieves 3.70 $\times$ over cuSPARSE on the RTX PRO 6000 and 4.09 $\times$ on the H200 (up to 146.61 $\times$). HSMU-SpGEMM achieves 0.74 $\times$ on the square matrices, and 0.32 $\times$ on the H200. On these matched inputs, SparseDitto is 5.37 $\times$ faster than HSMU-SpGEMM on the RTX PRO 6000 and 13.60 $\times$ faster on the H200. SparseDitto also covers the rectangular inputs that HSMU-SpGEMM does not support.

Panel (b) of both figures reports the per-matrix distribution. At $K = 8$ and $K = 32$, the whole SpMM distribution sits above cuSPARSE on both GPUs. As K grows, the box contracts toward the baseline. On the RTX PRO 6000, the interquartile range narrows from 3.28–5.57 $\times$ at $K = 8$ to 1.27–2.22 $\times$ at $K = 256$. SpGEMM has the widest spread on both platforms, with maxima of 146.61 $\times$ and 78.5 $\times$. This

variation is consistent with its sensitivity to intermediate-product distributions and accumulator design, beyond the input nonzero count alone.

4.2 Cache and Memory-Traffic Analysis

We use NVIDIA Nsight Compute to examine how the generated implementations use the memory hierarchy and GPU resources. We profile the timed kernels of each winning implementation, its cuSPARSE reference, and the specialized systems on the same inputs. The profiler replays each launch with flushed caches and locked base clocks. When a compute phase launches several kernels, we pool the sector counters over all launches. DRAM traffic is measured in bytes read and written at device memory. Table 1 summarizes cache hit rates and request counts, together with achieved occupancy and DRAM bandwidth.

Table 1. SparseDitto against each system on evaluated tasks. $\Delta L1$ and $\Delta L2$ are our hit rate minus theirs, in percentage points. The remaining columns are ours divided by theirs: L1 and L2 sector requests per operator invocation, achieved occupancy, and achieved DRAM bandwidth.

Comparison	$\Delta L1$	L1 req.	$\Delta L2$	L2 req.	Occ.	BW
vs. cuSPARSE	+4.7	1.09×	+3.6	0.77×	1.96×	1.98×
vs. CB-SpMV	+1.6	0.63×	-7.3	0.57×	0.93×	1.03×
vs. SparseTIR	-18.6	0.85×	-2.2	1.03×	1.24×	1.80×
vs. DTC-SpMM	-5.1	1.44×	-1.3	1.13×	3.21×	5.03×
vs. HSMU-SpGEMM	-16.3	0.83×	+1.5	0.97×	0.88×	4.25×

Versus cuSPARSE. SparseDitto issues slightly more L1 requests but fewer L2 requests, with higher hit rates at both levels. Its achieved occupancy and DRAM bandwidth are both about twice those of cuSPARSE. The gains thus accompany improvements in both memory access behavior and hardware utilization. The cache effects differ by operator. For SpMM, the median L1 hit-rate increase ranges from 8 to 17 percentage points across dense widths. For SpGEMM, the improvement appears primarily in L2.

Versus specialized systems. SparseTIR and HSMU-SpGEMM have higher L1 hit rates than SparseDitto, yet SparseDitto is 1.57× and 5.37× faster on the same inputs. Against SparseTIR, SparseDitto issues 0.85× as many L1 requests and 1.03× as many L2 requests, while sustaining 1.80× the DRAM bandwidth. Against CB-SpMV, it reduces L1 and L2 requests to 0.63× and 0.57×, despite a lower L2 hit rate.

Against HSMU-SpGEMM, SparseDitto achieves 0.88× the occupancy but 4.25× the DRAM bandwidth. The profiled execution uses 5 kernels per SpGEMM, compared with 24 for HSMU-SpGEMM. Fewer stages reduce launch overhead and the number of inter-kernel dependencies. This comparison highlights the importance of the accumulation dataflow and kernel decomposition, beyond cache hit rates or occupancy.

DTC-SpMM illustrates the interaction between representation overhead and hardware mapping. It moves slightly less data than SparseDitto, yet SparseDitto is 3.99× faster. SparseDitto achieves 3.21× its occupancy and 5.03× its DRAM bandwidth. DTC-SpMM’s Tensor-Core pipeline reaches only 1.5% of peak, and its 16×8 tiles have a median nonzero occupancy of 10.1% on these matrices. This low tile occupancy leaves much of the blocked computation operating on padding. Low achieved warp occupancy further limits latency hiding. SparseDitto’s advantage decreases from 6.40× at $K = 8$ to 2.89× at $K = 256$ as the dense width grows.

Together, these comparisons reveal that higher cache hit rates or occupancy alone do not guarantee better performance. SparseDitto outperforms baselines by covering broader optimization space instead of focusing on any single metrics.

4.3 Ablation Study and Comparison with General Agentic CUDA Coding Framework

We firstly compare with one SOTA general agentic CUDA coding framework StitchCUDA [15], whose workflow and prompts design are tailored for general CUDA tasks. It achieves 1.53×, 1.76×, and 0.86× over cuSPARSE on three operators separately. Then we evaluate three configurations with progressively richer sparse-specific guidance. *Learned Model + Agentic Lowering* generates implementations with guide from our learned cost model. The third augments the task with static pattern analysis as input to agentic lowering stage. The full configuration adds architecture-aware synthesis of compilation plans that coordinate representation, schedule, and hardware mapping.

Table 2 reports geometric-mean speedups for each configuration. *Learned Model + Agentic Lowering* already outperforms cuSPARSE on average. *Pattern Analysis* raises SpMV speedup from 1.93× to 2.27× and SpMM speedup from 2.02× to 2.28×. The larger gain is on SpGEMM, from 1.22× to 2.24×. The full configuration further improves all three operators, reaching 2.53×, 2.52×, and 3.70×, respectively. This progression supports sparse-specific guidance and architecture-aware plan synthesis significantly contribute to generated kernels’ performance.

Table 2. Ablation results by operator. Each cell reports the geometric-mean speedup over cuSPARSE.

Configuration	SpMV	SpMM	SpGEMM
StitchCUDA [15]	1.53×	1.76×	0.86×
Learned Model + Agentic Lowering	1.93×	2.02×	1.22×
* + Pattern Analysis	2.27×	2.28×	2.24×
* + Architecture-aware Synthesis (full)	2.53×	2.52×	3.70×

4.4 Learned Cost Model

We next evaluate the learned cost model’s ability to rank optimization templates and the structural features that influence

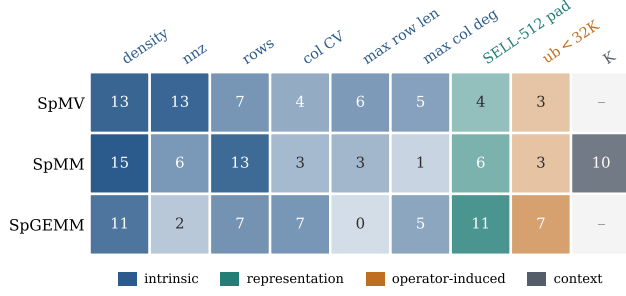


Figure 8. Structural feature importance per operator. Each cell gives a feature’s share of permutation importance in percent; colors group the feature families, and a dash marks a feature the operator does not use.

its ranking. This ranking supplies a prior for compilation-plan synthesis; the final implementation is selected through target-GPU measurements.

Training cost. The additive energy model has 15.9K parameters. Training on the full corpus of 400 matrices for 600 epochs takes about two seconds on a single RTX PRO 6000 or H200 GPU. At inference, ranking the templates takes well under a microsecond, a negligible cost relative to plan synthesis and agentic CUDA lowering.

Ranking quality. Our corpus contains 400 SuiteSparse matrices with measured template runtimes. It does not overlap with the evaluation set in Section 4.1. It also outperforms multilayer perceptron or gradient-boosted trees [9], while retaining an additive score decomposition. Full protocols, metric definitions, and per-operator results are provided in Appendix D.

Which features matter. Figure 8 reports permutation importance, which measures the model’s sensitivity to perturbing each feature across inputs. Matrix density, nonzero count, and row count are influential across all three operators. Operator-specific features also matter: the dense width K for SpMM and intermediate-product upper bounds for SpGEMM. These results support using both intrinsic pattern features and operator-dependent work estimates to rank templates. For an individual input, the additive score decomposition provides feature-level guidance to synthesis. Appendix B reports the complementary model-native importance measure.

4.5 End-to-End Application Evaluation

We evaluate whether SparseDitto’s specialized implementations improve application performance in a two-layer full-batch GCN on Reddit [10], following DTC-SpMM’s protocol [7]. We use an unweighted symmetric adjacency and sweep $K \in \{8, 32, 128, 256\}$. For each graph and dense width, SparseDitto generates the aggregation kernel once. The kernel can be reused across epochs and training runs on the

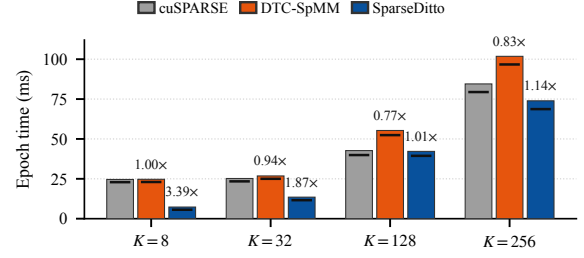


Figure 9. Full-batch GCN epoch time on Reddit across hidden widths. Horizontal marks inside bars denote SpMM time. Labels above DTC-SpMM and SparseDitto report speedup over cuSPARSE.

target GPU. Changes to node features and model parameters do not require regenerating it. Each result averages 100 epochs after 10 warmup epochs. Each epoch invokes four sparse aggregations: two in the forward pass and two in the backward pass. Symmetry also lets the backward pass reuse the corresponding forward kernel. Only the sparse aggregation implementation is replaced; the rest of the training loop is unchanged.

Figure 9 reports the result. At $K = 8$, SparseDitto reduces the average epoch time from 24.6 to 7.3 ms, yielding a 3.39× end-to-end speedup over cuSPARSE. At $K = 32$, it reduces the epoch from 25.2 to 13.5 ms and achieves 1.87×. The advantage narrows as the dense dimension grows, but SparseDitto remains faster than cuSPARSE. It reaches 1.01× at $K = 128$ and 1.14× at $K = 256$. DTC-SpMM matches cuSPARSE only at $K = 8$. Its speedups fall to 0.94×, 0.77×, and 0.83× at the other widths. SparseDitto is therefore 1.31×–3.38× faster than DTC-SpMM across the sweep.

4.6 Compilation and Lowering Cost

We report the one-time cost of constructing a specialized implementation, including plan synthesis, lowering, and target-GPU evaluation. SparseDitto takes about 30 minutes per matrix on average under the generation budget described above. LLM inference dominates this compilation time. Each task runs at most a few dozen compile-and-measure cycles, with each cycle taking seconds to tens of seconds depending on matrix size. The API cost for the main evaluation is about \$20 per matrix. These construction costs are separate from the execution times reported in Section 4.1.

AlphaSparse [6] similarly treats specialized kernel generation as an offline construction task. We allow its SpMV search a nine-hour budget per matrix. On Gset/G31, a graph matrix with 99% sparsity, it compiles and runs 730 candidate kernels within that budget. Its best kernel reaches 1.35× over cuSPARSE. SparseDitto reaches 4.46× on the same matrix in about 30 minutes.

5 Conclusion

The performance of sparse matrix operators on GPU depends jointly on the data representation, the execution schedule, and the hardware mapping, which need to all match the sparsity pattern. SparseDitto addresses this challenge with a unified compilation framework, covering SpMV, SpMM, and SpGEMM. Within this framework, an interpretable energy model ranks established strategy templates from structural features, an architecture-aware synthesis stage explores the design space under target-GPU constraints, and LLM agents automatically lowering this plan into CUDA code. SparseDitto outperforms cuSPARSE in evaluated tasks, achieving geometric-mean speedups of $2.68\times$ on an RTX PRO 6000 and $2.79\times$ on an H200. It runs $1.57\times$ to $5.37\times$ faster than four state-of-the-art specialized systems and accelerates full-batch GCN training by up to $3.39\times$.

References

- [1] Rishabh Agarwal, Levi Melnick, Nicholas Frosst, Xuezhou Zhang, Ben Lengerich, Rich Caruana, and Geoffrey E. Hinton. 2021. Neural additive models: Interpretable machine learning with neural nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 4699–4711.
- [2] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to rank using gradient descent. In *Proc. 22nd Int. Conf. on Machine Learning (ICML)*. 89–96.
- [3] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *Proc. 24th Int. Conf. on Machine Learning (ICML)*. 129–136.
- [4] Xing Cong, FuKai Sun, YiFan Chen, Chenhao Xie, Yi Liu, and Depei Qian. 2025. CB-SpMV: A Data Aggregating and Balance Algorithm for Cache-Friendly Block-Based SpMV on GPUs. In *Proceedings of the 39th ACM International Conference on Supercomputing*. 149–160.
- [5] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Software* 38, 1 (2011), 1:1–1:25.
- [6] Zhen Du, Jiajia Li, Yinshan Wang, Xueqi Li, Guangming Tan, and Ninghui Sun. 2022. AlphaSparse: generating high performance SpMV codes directly from sparse matrices. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (Dallas, Texas) (SC '22)*. IEEE Press, Article 66, 15 pages.
- [7] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 253–267. doi:10.1145/3620666.3651378
- [8] Salvatore Filippone, Valeria Cardellini, Davide Barbieri, and Alessandro Fanfarillo. 2017. Sparse Matrix-Vector Multiplication on GPGPUs. *ACM Trans. Math. Softw.* 43, 4, Article 30 (Jan. 2017), 49 pages. doi:10.1145/3017994
- [9] Jerome H. Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232.
- [10] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (Long Beach, California, USA) (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 1025–1035.
- [11] Trevor J. Hastie and Robert J. Tibshirani. 1990. *Generalized Additive Models*. Chapman & Hall/CRC.
- [12] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (Oct. 2017), 29 pages. doi:10.1145/3133901
- [13] Moritz Kreutzer, Georg Hager, Gerhard Wellein, Holger Fehske, and Alan R. Bishop. 2014. A Unified Sparse Matrix Data Format for Efficient General Sparse Matrix-Vector Multiplication on Modern Processors with Wide SIMD Units. 36, 5 (Jan. 2014), C401–C423. doi:10.1137/130930352
- [14] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu Jie Huang. 2006. A tutorial on energy-based learning. In *Predicting Structured Data*. MIT Press.
- [15] Shiyang Li, Zijian Zhang, Winson Chen, Yuebo Luo, Mingyi Hong, and Caiwen Ding. 2026. StitchCUDA: An Automated Multi-Agents End-to-End GPU Programming Framework with Rubric-based Agentic Reinforcement Learning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML'26)*.
- [16] Weifeng Liu and Brian Vinter. 2015. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing (Newport Beach, California, USA) (ICS '15)*. Association for Computing Machinery, New York, NY, USA, 339–350. doi:10.1145/2751205.2751209
- [17] Duane Merrill and Michael Garland. 2016. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 678–689. doi:10.1109/SC.2016.57
- [18] Yusuke Nagasaka, Akira Nukada, and Satoshi Matsuoka. 2017. High-Performance and Memory-Saving Sparse General Matrix-Matrix Multiplication for NVIDIA Pascal GPU. In *2017 46th International Conference on Parallel Processing (ICPP)*. 101–110. doi:10.1109/ICPP.2017.19
- [19] Yuyao Niu, Zhengyang Lu, Haonan Ji, Shuhui Song, Zhou Jin, and Weifeng Liu. 2022. TileSpGEMM: a tiled algorithm for parallel sparse general matrix-matrix multiplication on GPUs. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (Seoul, Republic of Korea) (PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 90–106. doi:10.1145/3503221.3508431
- [20] NVIDIA Corporation. 2026. cuSPARSE Library Documentation. <https://docs.nvidia.com/cuda/cusparse/>
- [21] OpenAI. 2026. GPT-5.6: Frontier Intelligence That Scales with Your Ambition. Accessed: 2026-07-31. <https://openai.com/index/gpt-5-6/>
- [22] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] <https://arxiv.org/abs/2502.10517>
- [23] Mathias Parger, Martin Winter, Daniel Mlakar, and Markus Steinberger. 2020. spECK: accelerating GPU sparse matrix-matrix multiplication through lightweight analysis (PPoPP '20). Association for Computing Machinery, New York, NY, USA, 362–375. doi:10.1145/3332466.3374521
- [24] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 683–698. doi:10.1145/3620665.3640426
- [25] Naser Sedaghati, Te Mu, Louis-Noël Pouchet, Srinivasan Parthasarathy, and P. Sadayappan. 2015. Automatic selection of sparse matrix representation on GPUs. In *Proc. 29th ACM Int. Conf. on Supercomputing (ICS)*.
- [26] Linghao Song, Yuze Chi, Atefeh Sohrabzadeh, Young-kyu Choi, Jason Lau, and Jason Cong. 2022. Sextans: A Streaming Accelerator for General-Purpose Sparse-Matrix Dense-Matrix Multiplication. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (Virtual Event, USA) (FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 65–77. doi:10.1145/3490422.3502357
- [27] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonese, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 766–780. doi:10.1109/MICRO50266.2020.00068
- [28] F. Vázquez, E.M. Garzón, and J.J. Fernández. 2010. A matrix approach to tomographic reconstruction and its implementation on GPUs. *Journal of Structural Biology* 170, 1 (2010), 146–151. doi:10.1016/j.jsb.2010.01.021
- [29] Yuke Wang, Boyuan Feng, Zheng Wang, Guyue Huang, and Yufei Ding. 2023. TC-GNN: Bridging Sparse GNN Computation and Dense Tensor Cores on GPUs. In *USENIX Annual Technical Conference (ATC)*.
- [30] Min Wu, Huizhang Luo, Fenfang Li, Yiran Zhang, Zhuo Tang, Kenli Li, Jeff Zhang, and Chubo Liu. 2025. HSMU-SpGEMM: Achieving High

- Shared Memory Utilization for Parallel Sparse General Matrix-Matrix Multiplication on Modern GPUs. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1452–1466.
- [31] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (*ASPLOS 2023*). Association for Computing Machinery, New York, NY, USA, 660–678. doi:10.1145/3582016.3582047
- [32] Yue Zhao, Jiajia Li, Chunhua Liao, and Xipeng Shen. 2018. Bridging the gap between deep learning and sparse matrix format selection. In *Proc. 23rd ACM SIGPLAN Symp. on Principles and Practice of Parallel Programming (PPoPP)*. 94–108.
- [33] Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, et al. 2025. QiMeng-Kernel: Macro-Thinking Micro-Coding Paradigm for LLM-Based High-Performance GPU Kernel Generation. *arXiv preprint arXiv:2511.20100* (2025).

A Compilation-Plan Design Space

Table 3 expands the compilation-plan formulation in Section 3. It lists representative choices for the representation $R = \langle L, I \rangle$, execution schedule $S = \langle P, D \rangle$, and hardware mapping θ_H of each operator. These choices are coupled: the representation constrains the parallel decomposition and dataflow, which in turn affect hardware resource use. Synthesis combines compatible choices into a complete plan. The entries describe optimization mechanisms, not a fixed set of complete kernels.

B Feature Importance

Model-native importance. The model in Equation 8 exposes each feature’s score contribution, $\varphi_{j,o,s}(x_j) = h_j(x_j)^T w_{2,o,j,s}$. Template rankings depend on differences between scores. We summarize a feature’s influence by the variation of its contribution across templates at the mean feature vector $\bar{x}$:

$$I_j(o) = \text{std}_{s \in V_o} \varphi_{j,o,s}(\bar{x}_j), \quad (16)$$

The values are normalized to sum to one per operator. Figure 10 reports this measure for the 36 structural features and the SpMM width K . It complements the permutation importance in Figure 8: one describes score variation at a reference feature vector, while the other measures sensitivity to feature permutation across inputs.

Density, nonzero count, and row count are influential across operators. The dense width K is prominent for SpMM, while intermediate-work features matter for SpGEMM. About 30 features account for 90% of the model-native importance, indicating that the scores draw on a broad set of structural signals.

C Evaluation Matrices

Table 4 lists the 60 SuiteSparse matrices used in Section 4, sorted by nonzero count. Mean and Max denote the mean and maximum number of nonzeros per row. CV is the coefficient of variation of row length, and density is $\text{nnz}(X)/(mn)$. The last column indicates whether the matrix is square.

D Learned Cost Model Evaluation

This section supplements Section 4.4 with the evaluation protocol, metric definitions, and per-operator results for the learned template-ranking model.

Protocol. The offline corpus contains 400 SuiteSparse matrices with measured template runtimes. Because the structural features of a matrix are shared across its tasks, we report five-fold cross-validation grouped by matrix and stratified by shape, averaged over five seeds; a separate 100-matrix test set is held out and never used for tuning. We use four metrics per operator. *Top-1* is the fraction of inputs whose top-ranked template is the measured best. *Regret* is the geometric-mean slowdown of the top-ranked template relative to the per-input oracle. *Capture* is the fraction of the oracle speedup

	SpMV	SpMM	SpGEMM
density	13	15	11
nnz	13	6	2
rows	7	13	7
col CV	4	3	7
max row len	6	3	0
max col deg	5	1	5
avg row len	3	3	5
empty rows	2	2	4
col gap	4	2	1
row p75	3	2	1
row p25	1	1	3
bandwidth	1	3	2
tile nnz share	3	1	2
avg col deg	1	1	2
tile cover	1	1	1
bw ratio	1	1	0
row CV	1	1	0
diag dom	1	0	0
SELL-512 pad	4	6	11
BSR-4 fill	2	2	6
BSR-16 fill	2	3	1
BSR-8 fill	1	2	2
SELL-32 pad	1	2	1
ELL pad	2	1	0
16x16 fill	0	1	1
SELL-16 pad	1	0	1
ub < 32K	3	3	7
row-prod p95	3	2	6
AA ^T ub	3	5	3
row-prod max	3	2	1
ub < 4K	2	1	1
compress	1	1	2
AA ^T ub CV	1	0	1
ub CV	1	1	1
ub < 512	1	0	0
Σ products	0	0	1
K	–	10	–

■ intrinsic
■ operator-induced

■ representation
■ context

Figure 10. Normalized model-native feature importance by operator, in percent. Features are grouped by the analysis taxonomy and ordered within each group. The dense width K applies only to SpMM.

realized by the ranking. *Cliff* is the fraction of inputs whose top choice is more than $2\times$ slower than the CSR baseline. Regret and capture, rather than top-1, are the quantities that reflect the performance cost of a ranking error, because selecting a near-optimal template can preserve performance despite a top-1 error.

Baselines. We compare against a multilayer perceptron (MLP) ranker trained with the same listwise objective, and gradient-boosted trees (GBDT), a strong reference model for tabular data. All models use the same 36 structural features and identical cross-validation folds. *Energy (listwise)* ablates our model by removing the margin term ($\lambda_o = 0$), leaving the same architecture trained as a listwise ranker.

Ranking quality. The energy model improves on the MLP in top-1 accuracy, regret, and capture for SpMM and SpGEMM (Table 5). Capture rises from 76% to 81% for SpMM

Op.	Layout and metadata $\langle L, I \rangle$	Parallel decomposition P	Dataflow D	Hardware mapping θ_H
SpMV	CSR, COO, ELL, SELL- C - σ with slice height C and sort window σ , BSR, adaptive 16×16 tiles with per-tile format selection. Metadata: row pointers, slice offsets, block indices, row permutations, per-tile descriptors.	One thread, one warp, or one CTA per row; equal-nonzero chunks with merge-path boundaries; slice- or block-parallel; cyclic row assignment for balance.	Direct write to y ; segmented reduction across chunk boundaries; atomic accumulation; read-only or texture-cached access to x ; windowed staging of x segments in shared memory.	Rows or nonzeros per CTA, slice height, block size, threads per block, active column-window width, unroll factor.
SpMM	CSR, COO, BSR, Blocked-ELL, packed column windows, adaptive tiles. Metadata: row and block indices, window descriptors, Tensor-Core fragment maps.	Sparse rows, nonzeros, or blocks crossed with tiles of the dense dimension K ; one warp per row window; two-level tiling over (rows, K).	Reuse of dense rows across sparse nonzeros; accumulation in registers or shared memory; B staged in shared memory with C held in registers; CUDA-core or Tensor-Core (mma) execution; double buffering of the sparse stream.	K tile width, block shape, warps per CTA, shared-memory staging size, fragment layout, double-buffer depth.
SpGEMM	CSR, row-reordered CSR, tiled inputs, materialized A^T for AA^T . Metadata: row bins by upper-bound work, intermediate-product counts, tile pointers.	Rows grouped into bins by predicted output size; one warp or CTA per row; tile-parallel; separate symbolic and numeric phases.	Dense, hash, sorted-merge, or hybrid accumulation; on-chip accumulators sized per bin; global spill for oversized rows; output compaction by prefix sum.	Hash-table capacity, dense-stripe width, bin thresholds, threads per row, shared-memory allocation per CTA.

Table 3. Representative compilation-plan choices by operator. Synthesis coordinates representation, parallel decomposition, dataflow, and hardware mapping rather than selecting them independently.

and from 76% to 79% for SpGEMM. For SpMV, both models have regret 1.03; the energy model improves capture from 95% to 96%, with slightly lower top-1 accuracy.

GBDT achieves lower regret and higher capture on SpMM and SpGEMM. The additive model remains competitive in top-1 accuracy on SpMV and SpMM while exposing feature-level score contributions to synthesis. Cliff rates remain below 6% for all models. The energy model’s rates are 0.0% for SpMV and 0.5% for SpMM, no higher than the MLP’s. Removing the margin term reduces top-1 accuracy and capture on SpMM and SpGEMM. This supports weighting pairwise rankings by measured speedup gaps.

E Case Studies

The following cases illustrate how input structure and target hardware affect the generated implementation. Table 6 reports two tasks per operator on the RTX PRO 6000 Blackwell GPU with CUDA 13.0. The SpMV and SpMM pairs contrast different row-length distributions; the SpGEMM pair contrasts intermediate work and output size. A final case compares the same SpMM task on both GPUs. Speedups use cuSPARSE unless explicitly noted.

E.1 SpMV

JGD_Homology_ch7-9-b5: uniform row lengths. This $423,360 \times 317,520$ matrix has exactly six nonzeros in every row, with CV 0.00. ELL therefore introduces no padding. The generated plan uses column-major ELL and assigns one thread per row. Adjacent threads access the same slot in adjacent

rows through consecutive addresses. The selector initially ranks SELL with slice height 32 first. Synthesis instead produces an ELL implementation that avoids slice metadata and achieves a $2.44\times$ speedup over cuSPARSE.

Barabasi_NotreDame_actors: skewed rows. The input contains 392,400 rows and 1.47M nonzeros. The mean row length is 3.7, the maximum is 646, and CV is 2.75. Padding every row to the maximum would increase storage by roughly two orders of magnitude. The generated implementation instead uses COO and partitions the nonzero stream into equal-size chunks, with one nonzero per thread. Warp-local segmented reductions combine contributions to the same row, and atomic additions merge partial sums across warps. This balances nonzero processing across warps and achieves $1.94\times$. The two SpMV cases illustrate how row-length variation changes both representation and parallel decomposition.

E.2 SpMM

Schenk_AFE_af_shell10 at $K=32$: regular finite-element structure. This matrix has 1.5M rows and 52.7M nonzeros, with mean row length 34.9, maximum 35, and CV 0.03. ELL requires little padding. The generated kernel assigns a four-thread group to each sparse row. Each thread accumulates eight output values in registers, while shuffle instructions share sparse entries within the group. The implementation pipelines sparse loads and writes each output once the row is complete. It achieves $6.82\times$. Although the selector ranks a Tensor-Core block template first, measured refinement selects this CUDA-core implementation. As in Section 4.2,

Table 4. The 60 evaluation matrices, sorted by nonzero count.

Matrix	Rows	Cols	NNZ	Density	Mean	Max	CV	Sq.
HB_bcsstm19	817	817	817	1.2×10^{-3}	1.0	1	0.00	yes
HB_str_0	363	363	2,454	1.9×10^{-2}	6.8	34	1.09	yes
LPnetlib_lp_standata	359	1,274	3,230	7.1×10^{-3}	9.0	745	4.41	no
HB_bp_600	822	822	4,172	6.2×10^{-3}	5.1	302	2.37	yes
Bai_cdde2	961	961	4,681	5.1×10^{-3}	4.9	5	0.07	yes
Sandia_fpga_dcop_04	1,220	1,220	5,884	4.0×10^{-3}	4.8	36	0.83	yes
Sandia_fpga_dcop_11	1,220	1,220	5,892	4.0×10^{-3}	4.8	36	0.83	yes
Sandia_fpga_dcop_47	1,220	1,220	5,892	4.0×10^{-3}	4.8	36	0.83	yes
JGD_Homology_n3c6-b10	675	2,511	7,425	4.4×10^{-3}	11.0	11	0.00	no
Oberwolfach_flowmeter0	9,669	9,669	9,669	1.0×10^{-4}	1.0	1	0.00	yes
HB_can_838	838	838	10,010	1.4×10^{-2}	11.9	32	0.52	yes
Sandia_adder_dcop_44	1,813	1,813	11,245	3.4×10^{-3}	6.2	1,310	4.96	yes
Sandia_adder_dcop_54	1,813	1,813	11,246	3.4×10^{-3}	6.2	1,310	4.96	yes
PARSEC_Si2	769	769	17,801	3.0×10^{-2}	23.1	61	0.39	yes
HB_bcsstk09	1,083	1,083	18,437	1.6×10^{-2}	17.0	23	0.24	yes
Gset_G47	1,000	1,000	19,980	2.0×10^{-2}	20.0	34	0.22	yes
JGD_G5_IG5-11	1,227	1,692	22,110	1.1×10^{-2}	18.0	111	0.83	no
Newman_hep-th	8,361	8,361	31,502	4.5×10^{-4}	3.8	50	1.14	yes
Gset_G31	2,000	2,000	39,980	10.0×10^{-3}	20.0	40	0.23	yes
JGD_Franz_Franz5	7,382	2,882	44,056	2.1×10^{-3}	6.0	6	0.04	no
DIMACS10_fe_4elt2	11,143	11,143	65,636	5.3×10^{-4}	5.9	12	0.15	yes
FIDAP_ex14	3,251	3,251	66,775	6.3×10^{-3}	20.5	37	0.38	yes
Rommens_bips07_3078	21,128	21,128	75,729	1.7×10^{-4}	3.6	27	0.76	yes
GHS_indef_aug2d	29,008	29,008	76,832	9.1×10^{-5}	2.6	4	0.36	yes
Schenk_IBMNA_c-38	8,127	8,127	77,689	1.2×10^{-3}	9.6	889	1.73	yes
Boeing_msc04515	4,515	4,515	97,707	4.8×10^{-3}	21.6	27	0.23	yes
FIDAP_ex15	6,867	6,867	98,671	2.1×10^{-3}	14.4	18	0.30	yes
Hollinger_g7jac040	11,790	11,790	114,671	8.2×10^{-4}	9.7	120	1.50	yes
Bindel_ted_B_unscaled	10,605	10,605	144,579	1.3×10^{-3}	13.6	49	0.89	yes
ML_Graph_kmnist_norm_10NN	10,000	10,000	156,932	1.6×10^{-3}	15.7	140	0.47	yes
Schenk_IBMNA_c-50	22,401	22,401	193,625	3.9×10^{-4}	8.6	1,919	2.65	yes
Rajat_rajat22	39,899	39,899	197,264	1.2×10^{-4}	4.9	3,401	5.05	yes
Toledo_deltaX	68,600	21,961	247,424	1.6×10^{-4}	3.6	83	1.63	no
JGD_Franz_Franz11	47,104	30,144	329,728	2.3×10^{-4}	7.0	7	0.00	no
JGD_GL7d_GL7d13	47,271	8,899	356,232	8.5×10^{-4}	7.5	14	0.34	no
Quaglino_viscoplastic2	32,769	32,769	381,326	3.6×10^{-4}	11.6	73	1.20	yes
Andrianov_net25	9,520	9,520	401,200	4.4×10^{-3}	42.1	139	0.77	yes
GHS_indef_c-59	41,282	41,282	480,536	2.8×10^{-4}	11.6	3,090	2.04	yes
Shen_e40r0100	17,281	17,281	553,562	1.9×10^{-3}	32.0	62	0.49	yes
JGD_Homology_ch7-8-b4	141,120	58,800	705,600	8.5×10^{-5}	5.0	5	0.00	no
AMD_G2_circuit	150,102	150,102	726,674	3.2×10^{-5}	4.8	6	0.13	yes
Watson_Baumann	112,211	112,211	760,631	6.0×10^{-5}	6.8	7	0.06	yes
QY_case39	40,216	40,216	1,042,160	6.4×10^{-4}	25.9	20,024	12.20	yes
Pereyra_landmark	71,952	2,704	1,151,232	5.9×10^{-3}	16.0	16	0.00	no
Norris_heart1	3,557	3,557	1,387,773	1.1×10^{-1}	390.2	1,120	0.32	yes
DIMACS10_ga2010	291,086	291,086	1,418,056	1.7×10^{-5}	4.9	85	0.61	yes
CEMW_vfem	93,476	93,476	1,434,636	1.6×10^{-4}	15.3	28	0.20	yes
Barabasi_NotreDame_actors	392,400	127,823	1,470,404	2.9×10^{-5}	3.7	646	2.75	no
Nemeth_nemeth26	9,506	9,506	1,511,760	1.7×10^{-2}	159.0	193	0.16	yes
JGD_Homology_ch7-9-b5	423,360	317,520	2,540,160	1.9×10^{-5}	6.0	6	0.00	no
Andrianov_net125	36,720	36,720	2,577,200	1.9×10^{-3}	70.2	231	0.95	yes
TSOPF_TSOPF_FS_b300_c2	56,814	56,814	8,767,466	2.7×10^{-3}	154.3	27,742	6.23	yes
Gupta_gupta3	16,783	16,783	9,323,427	3.3×10^{-2}	555.5	14,672	2.22	yes
LAW_in-2004	1,382,908	1,382,908	16,917,053	8.8×10^{-6}	12.2	7,753	3.04	yes
Janna_CoupCons3D	416,800	416,800	22,322,336	1.3×10^{-4}	53.6	76	0.16	yes
Belcastro_human_gene1	22,283	22,283	24,669,643	5.0×10^{-2}	1107.1	7,939	1.27	yes
DIMACS10_asia_osm	11,950,757	11,950,757	25,423,206	1.8×10^{-7}	2.1	9	0.23	yes
Belcastro_mouse_gene	45,101	45,101	28,967,291	1.4×10^{-2}	642.3	8,032	1.33	yes
Schenk_AFE_af_shell10	1,508,065	1,508,065	52,672,325	2.3×10^{-5}	34.9	35	0.03	yes
DIMACS10_delaunay_n24	16,777,216	16,777,216	100,663,202	3.6×10^{-7}	6.0	26	0.22	yes

template rankings provide an initial prior rather than fixing the final implementation.

LAW_in-2004 at $K=128$: a web graph. This matrix has 1.4M rows and 16.9M nonzeros, with mean row length 12.2, maximum 7,753, and CV 3.04. The generated implementation separates short rows from long rows. A warp processes

each short row with register accumulation. Long rows are split into chunks of at most 256 nonzeros, each assigned to a CTA; atomic additions combine the partial outputs. Both paths read the dense operand directly. This hybrid mapping achieves 1.97 $\times$, with a compute time of 1.874 ms. Preprocessing takes 8.9 s and is excluded from that measurement. Its

Model	SpMV			SpMM			SpGEMM		
	top1↑	regret↓	capt.↑	top1↑	regret↓	capt.↑	top1↑	regret↓	capt.↑
MLP	0.805	1.03	95%	0.715	1.17	76%	0.681	1.37	76%
GBDT [‡]	0.793	1.03	96%	0.716	1.09	87%	0.713	1.26	82%
Energy (listwise)	0.797	1.03	96%	0.716	1.14	80%	0.697	1.34	78%
Energy (ours)	0.797	1.03	96%	0.719	1.13	81%	0.702	1.32	79%

Table 5. Template-ranking quality (5-fold grouped cross-validation, averaged over 5 seeds). [‡] marks the black-box reference model. Bold marks where the energy model beats the MLP on SpMM and SpGEMM. Cliff rates are discussed in the text.

Op.	Matrix	Mean	Max	CV	Kernel (ms)	Speedup
SpMV	JGD_Homology_ch7-9-b5	6.0	6	0.00	0.007	2.44×
SpMV	Barabasi_NotreDame_actors	3.7	646	2.75	0.013	1.94×
SpMM	Schenk_AFE_af_shell10 ($K=32$)	34.9	35	0.03	0.736	6.82×
SpMM	LAW_in-2004 ($K=128$)	12.2	7,753	3.04	1.874	1.97×
SpGEMM	DIMACS10_asia_osm	2.1	9	0.23	3.771	6.45×
SpGEMM	DIMACS10_delaunay_n24	6.0	26	0.22	22.009	1.19×

Table 6. Case-study tasks. Mean and Max are nonzeros per row, CV their coefficient of variation. Speedup is over cuSPARSE, except [†]asia_osm, where cuSPARSE SpGEMM exits with an out-of-resources error and the reference is the HSMU-SpGEMM FP64 build instead.

work lists depend on the sparse structure and can be reused across subsequent calls with different dense operands. Appendix F gives the generated source.

E.3 SpGEMM

DIMACS10_asia_osm: a sparse road network. This road network has 12.0M rows, 2.1 nonzeros per row on average, and density 1.8×10^{-7} . The product AA has 42.8M nonzeros. A full-width dense accumulator would require about 12M entries per row despite the sparse output. The generated plan instead bins rows by their intermediate-product upper bounds. Rows with bounds of at most 8 or 36 use fixed-capacity thread-local accumulators; larger rows use a separate traversal-based path. The measured speedup is 6.45× against the HSMU-SpGEMM FP64 build. cuSPARSE exits with an out-of-resources error on this task, so this comparison is separate from the main cuSPARSE speedup aggregate.

DIMACS10_delaunay_n24: a large sparse output. This Delaunay triangulation has 16.8M rows and 100.7M nonzeros, with six nonzeros per row on average. The generated plan bins rows by their intermediate-product upper bounds and assigns a warp to each row. The two bins use shared-memory hash tables with 64 and 512 entries, respectively. A compaction pass produces the final output, which contains 347M nonzeros. The kernel achieves 1.19×. Binning reduces accumulator overhead, but it does not remove the substantial traffic required to materialize this output.

E.4 The Same Task on Two GPUs

The final case fixes the matrix and varies the GPU to illustrate architecture-aware synthesis.

Nemeth_nemeth26 at $K=8$ is a 9,506-row matrix with a strongly diagonal structure. It has 159 nonzeros per row on average with CV 0.16, a median row bandwidth of 185 columns, and an average gap of 1.2 between consecutive column indices. Dense tiles account for 65.2% of the nonempty 16×16 tiles and contain 88.2% of the nonzeros. The structural profile and selector ranking are identical on both GPUs, with a 32×32 Blocked-ELL Tensor-Core template ranked first.

On the RTX PRO 6000, the selected implementation uses Tensor Cores and achieves 4.57× over cuSPARSE. On the H200, synthesis retains a Tensor-Core candidate but also explores a SIMT block-row plan. The latter wins at 8.33×.

The planning record illustrates how hardware capacities guide these choices. It estimates a 9.28 MiB working set for a 16×16 BSR candidate, including dense operands and output, compared with 12.15 MiB for CSR. It compares these footprints with the H200’s 50 MiB physical L2 capacity and considers an 8 KiB shared-memory tile for a windowed candidate. These estimates guide initial plans; actual cache behavior and parallelism depend on the lowered code. Target-GPU measurements then determine which implementation to retain. The different winners show why an identical structural prior need not lead to an identical implementation across GPUs.

F A Generated SpMM Implementation

Listing 1 gives the generated SpMM implementation for LAW_in-2004 at $K=128$, without manual edits. It follows the harness’s preprocessing, compute, and cleanup interface. Preprocessing prepares the sparse representation and work lists. The compute entry point evaluates the operator, and cleanup releases resources. The reported $1.97\times$ speedup measures only the compute function, as in the main evaluation.

Preprocessing separates rows shorter than 256 nonzeros from chunks of longer rows. The short-row kernel assigns one warp per row, with four output accumulators per thread. The long-row kernel assigns a 128-thread CTA to each chunk and atomically combines partial outputs. Both kernels read CSR and the dense operand directly. The generated preprocessing also constructs a windowed representation, but the compute function does not use it. The executed plan combines row-wise and chunk-wise parallelism within one implementation.

Listing 1. Generated SpMM implementation for LAW_in-2004 at $K=128$.

```

1  #include <cuda_runtime.h>
2  #include <stdint.h>
3  #include <vector>
4  #include "sparse_formats.cuh"
5
6  struct SpmmContext {
7      WindowedData windowed;
8      int* d_light_rows;
9      int light_count;
10     int* d_chunk_rows;
11     int* d_chunk_begin;
12     int* d_chunk_end;
13     int chunk_count;
14     int num_rows;
15     int nnz;
16 };
17
18 __global__ __launch_bounds__(256, 4)
19 void light_rows_spmm_kernel(
20     const int* __restrict__ row_ids,
21     int count,
22     const int* __restrict__ row_ptr,
23     const int* __restrict__ col_ind,
24     const float* __restrict__ values,
25     const float* __restrict__ B,
26     float* __restrict__ C)
27 {
28     const int tid = (int)threadIdx.x;
29     const int warp = tid >> 5;
30     const int lane = tid & 31;
31     const int idx = (int)blockIdx.x * 8 + warp;
32
33     if (idx >= count) return;
34
35     const int row = row_ids[idx];
36     const int begin = row_ptr[row];
37     const int end = row_ptr[row + 1];
38
39     float x0 = 0.0f;
40     float x1 = 0.0f;
41     float x2 = 0.0f;
42     float x3 = 0.0f;
43
44     for (int p = begin; p < end; ++p) {
45         const int col = __ldg(col_ind + p);
46         const float a = __ldg(values + p);
47         const float* bp = B + ((size_t)col << 7) + lane;
48
49         x0 = fmaf(a, __ldg(bp), x0);
50         x1 = fmaf(a, __ldg(bp + 32), x1);
51         x2 = fmaf(a, __ldg(bp + 64), x2);
52         x3 = fmaf(a, __ldg(bp + 96), x3);
53     }
54
55     float* cp = C + ((size_t)row << 7) + lane;
56     cp[0] = x0;
57     cp[32] = x1;
58     cp[64] = x2;
59     cp[96] = x3;
60 }
61
62 __global__ __launch_bounds__(128, 6)
63 void compact_dense_chunks_spmm_kernel(
64     const int* __restrict__ chunk_rows,
65     const int* __restrict__ chunk_begin,
66     const int* __restrict__ chunk_end,
67     int chunk_count,
68     const int* __restrict__ col_ind,
69     const float* __restrict__ values,
70     const float* __restrict__ B,
71     float* __restrict__ C)
72 {
73     const int task = (int)blockIdx.x;
74     if (task >= chunk_count) return;
75
76     const int lane = (int)threadIdx.x;
77     const int row = chunk_rows[task];
78     const int begin = chunk_begin[task];
79     const int end = chunk_end[task];
80
81     float s = 0.0f;
82
83     for (int p = begin; p < end; ++p) {
84         const int col = __ldg(col_ind + p);
85         const float a = __ldg(values + p);
86         s = fmaf(a, __ldg(B + ((size_t)col << 7) + lane), s);
87     }
88
89     atomicAdd(C + ((size_t)row << 7) + lane, s);
90 }
91
92 void* spmm_preprocess(
93     const int* d_row_ptr,
94     const int* d_col_ind,
95     const float* d_values,
96     int num_rows,
97     int num_cols,
98     int nnz,
99     int K)
100 {
101     (void)K;
102
103     SpmmContext* ctx = new SpmmContext;
104     ctx->d_light_rows = nullptr;
105     ctx->d_chunk_rows = nullptr;
106     ctx->d_chunk_begin = nullptr;
107     ctx->d_chunk_end = nullptr;
108     ctx->light_count = 0;
109     ctx->chunk_count = 0;
110     ctx->num_rows = num_rows;
111     ctx->nnz = nnz;
112
113     ctx->windowed = csr_to_windowed(
114         d_row_ptr, d_col_ind, d_values,

```

```

115     num_rows, num_cols, nnz, 256, 4);
116
117     if (num_rows <= 0)
118         return ctx;
119
120     std::vector<int> h_row_ptr((size_t)num_rows + 1);
121     cudaMemcpy(h_row_ptr.data(), d_row_ptr,
122               ((size_t)num_rows + 1) * sizeof(int),
123               cudaMemcpyDeviceToHost);
124
125     std::vector<int> h_light_rows;
126     std::vector<int> h_chunk_rows;
127     std::vector<int> h_chunk_begin;
128     std::vector<int> h_chunk_end;
129
130     h_light_rows.reserve(num_rows);
131
132     const int chunk_nnz = 256;
133     for (int r = 0; r < num_rows; ++r) {
134         const int begin = h_row_ptr[r];
135         const int end = h_row_ptr[r + 1];
136         const int n = end - begin;
137
138         if (n < chunk_nnz) {
139             h_light_rows.push_back(r);
140         } else {
141             for (int p = begin; p < end; p += chunk_nnz) {
142                 int q = p + chunk_nnz;
143                 if (q > end) q = end;
144                 h_chunk_rows.push_back(r);
145                 h_chunk_begin.push_back(p);
146                 h_chunk_end.push_back(q);
147             }
148         }
149     }
150
151     ctx->light_count = (int)h_light_rows.size();
152     ctx->chunk_count = (int)h_chunk_rows.size();
153
154     if (ctx->light_count > 0) {
155         cudaMalloc(&ctx->d_light_rows,
156                   (size_t)ctx->light_count * sizeof(int));
157         cudaMemcpy(ctx->d_light_rows, h_light_rows.data(),
158                   (size_t)ctx->light_count * sizeof(int),
159                   cudaMemcpyHostToDevice);
160     }
161
162     if (ctx->chunk_count > 0) {
163         const size_t bytes = (size_t)ctx->chunk_count * sizeof(int);
164         cudaMalloc(&ctx->d_chunk_rows, bytes);
165         cudaMalloc(&ctx->d_chunk_begin, bytes);
166         cudaMalloc(&ctx->d_chunk_end, bytes);
167
168         cudaMemcpy(ctx->d_chunk_rows, h_chunk_rows.data(),
169                   bytes, cudaMemcpyHostToDevice);
170         cudaMemcpy(ctx->d_chunk_begin, h_chunk_begin.data(),
171                   bytes, cudaMemcpyHostToDevice);
172         cudaMemcpy(ctx->d_chunk_end, h_chunk_end.data(),
173                   bytes, cudaMemcpyHostToDevice);
174     }
175
176     return ctx;
177 }
178
179 void spmm_compute(
180     void* context,
181     const int* d_row_ptr,
182     const int* d_col_ind,
183     const float* d_values,
184     const float* d_B,

```

```

185     float* d_C,
186     int num_rows,
187     int num_cols,
188     int nnz,
189     int K)
190 {
191     (void)num_cols;
192     (void)nnz;
193     (void)K;
194
195     SpmmContext* ctx = static_cast<SpmmContext*>(context);
196     if (!ctx || num_rows <= 0)
197         return;
198
199     if (ctx->nnz == 0) {
200         cudaMemsetAsync(d_C, 0, (size_t)num_rows * 128 * sizeof(
201             float), 0);
202         return;
203     }
204
205     if (ctx->chunk_count > 0) {
206         cudaMemsetAsync(d_C, 0, (size_t)num_rows * 128 * sizeof(
207             float), 0);
208
209         if (ctx->light_count > 0) {
210             const int blocks = (ctx->light_count + 7) >> 3;
211             light_rows_spmm_kernel<<<blocks, 256>>>(&
212                 ctx->d_light_rows,
213                 ctx->light_count,
214                 d_row_ptr,
215                 d_col_ind,
216                 d_values,
217                 d_B,
218                 d_C);
219
220             compact_dense_chunks_spmm_kernel<<<ctx->chunk_count,
221                 128>>>(&
222                 ctx->d_chunk_rows,
223                 ctx->d_chunk_begin,
224                 ctx->d_chunk_end,
225                 ctx->chunk_count,
226                 d_col_ind,
227                 d_values,
228                 d_B,
229                 d_C);
230
231             if (ctx->light_count > 0) {
232                 const int blocks = (ctx->light_count + 7) >> 3;
233                 light_rows_spmm_kernel<<<blocks, 256>>>(&
234                     ctx->d_light_rows,
235                     ctx->light_count,
236                     d_row_ptr,
237                     d_col_ind,
238                     d_values,
239                     d_B,
240                     d_C);
241             }
242         }
243     }
244
245     void spmm_cleanup(void* context)
246     {
247         SpmmContext* ctx = static_cast<SpmmContext*>(context);
248         if (!ctx)
249             return;
250
251         if (ctx->d_light_rows)
252             cudaFree(ctx->d_light_rows);
253         if (ctx->d_chunk_rows)
254             cudaFree(ctx->d_chunk_rows);
255         if (ctx->d_chunk_begin)
256             cudaFree(ctx->d_chunk_begin);

```

SparseDitto: An Agentic Sparse Compilation Framework through Architecture-Aware Synthesis on GPUs

```
253     if (ctx->d_chunk_end)
254         cudaFree(ctx->d_chunk_end);
255
256     free_windowed(ctx->windowed);
257     delete ctx;
258 }
```